

Summer Reading List: Operating Systems & File Systems

Stony Brook University
Graduate Systems Reading Program

Summer 2026

Abstract

This curated bibliography compiles 26 seminal and modern academic papers covering foundational operating systems, innovative kernel architectures, local and distributed file systems, virtualization, and hardware-software co-design. It includes classic texts essential for graduate studies alongside prominent research originating from Stony Brook University labs.

Part I: Foundational & Monolithic OS Architectures

These classic papers established fundamental abstractions such as processes, threads, virtual memory, and unified resource interfaces.

1. The UNIX Time-Sharing System

Dennis M. Ritchie and Ken Thompson (1974)

The foundational text for UNIX. Introduces the paradigm of treating "everything as a file," hierarchical file systems, and shell pipelines.

☑

2. The Nucleus of a Multiprogramming System

Per Brinch Hansen (1970)

An early, highly influential exploration of separating operating system policy from mechanism, serving as a philosophical precursor to microkernels.

☐

3. The Protection of Information in Computer Systems

Jerome H. Saltzer and Michael D. Schroeder (1975)

The definitive security architecture paper. Establishes timeless design principles including "least privilege" and "fail-safe defaults."

☐

4. Hints for Computer System Design

Butler W. Lampson (1983)

A distillation of practical wisdom for system builders, championing principles like "keep it simple," "use hints," and "plan to throw one away."

☐

Part II: Kernel Design Evolutions (Microkernels & Extensibility)

Key milestones in the historical structural debate regarding whether the kernel should be monolithic or a minimal IPC broker.

5. The Performance of Micro-Kernel Implementations

Jochen Liedtke (1993)

The landmark paper that revitalized microkernel research by introducing L4 and demonstrating that IPC bottlenecks were implementation flaws, not architectural limitations.

□

6. Extensibility, Safety and Performance in the SPIN Operating System

Brian N. Bershad, Stefan Savage, Pardha Pratim, Emin Gün Sirer, Marc E. Fiuczynski, David Becker, Craig Chambers, and Susan Eggers (1995)

Explored dynamically downloading user code safely into kernel space via type-safe languages (Modula-3), serving as a spiritual ancestor to eBPF.

□

7. Exokernel: An Operating System Architecture for Application-Specific Resource Management

Dawson R. Engler, M. Frans Kaashoek, and James O'Toole Jr. (1995)

Advocated for eliminating standard abstractions to give untrusted applications low-level, direct access to physical hardware resources.

□

Part III: Classical and Local File Systems

The evolution of persistent storage mapping physical layouts to handle physical mechanical constraints and data corruption.

8. A Fast File System for UNIX

Marshall Kirk McKusick, William N. Joy, Samuel J. Leffler, and Robert S. Fabry (1984)

The classic BSD FFS paper detailing how optimizing block layout into "cylinder groups" drastically improved physical disk locality and bandwidth.

□

9. The Design and Implementation of a Log-Structured File System

Mendel Rosenblum and John K. Ousterhout (1992)

Introduced LFS, treating disk storage as a continuous, append-only log to optimize write throughput; highly applicable to modern Flash Translation Layers (FTL).

□

10. Analysis and Evolution of Journaling File Systems

Vijayan Prabhakaran, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau (2005)

An exhaustive analysis of transactional logging variants (data vs. metadata journaling) under failure states to preserve consistency.

□

Part IV: Distributed Systems & Network File Systems

Architectures adapting state, caching, consistency protocols, and scale across network boundaries.

11. Scale and Performance in a Distributed File System

John H. Howard, Michael J. Kazar, Sherri G. Menees, David A. Nichols, Mahadev Satyanarayanan, Robert N. Sidebotham, and Michael J. West (1988)

Introduced the Andrew File System (AFS), detailing aggressive client-side whole-file caching and stateful callback architectures.

□

12. **The Google File System**

Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung (2003)

Pioneered warehouse-scale storage by assuming commodity hardware failure is the norm, optimizing for multi-gigabyte, append-only workloads.

□

13. **Bigtable: A Distributed Storage System for Structured Data**

Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber (2006)

A wide-column, highly scalable database layer built on top of GFS using Log-Structured Merge-trees (LSM-trees).

□

14. **Ceph: A Scalable, High-Performance Distributed File System**

Sage A. Weil, Scott A. Brandt, Ethan L. Miller, Darrell D. E. Long, and Carlos Maltzahn (2006)

Decoupled storage metadata maps via the algorithmic CRUSH pseudo-random data placement function, removing structural centralized lookup tables.

□

Part V: Virtualization, Concurrency, & Modern Paradigms

Adapting operating systems to massive core scaling, direct memory buses, and single-tenant cloud execution contexts.

15. **SOSP's Do's and Don'ts: The Virtual Machine Monitor**

Mendel Rosenblum and Tal Garfinkel (2005)

A retrospective and forward-looking text outlining the practical industrial rebirth of full virtualization and hypervisors.

□

16. **Xen and the Art of Virtualization**

Paul Barham, Boris Dragovic, Keir Fraser, Steven Hand, Tim Harris, Alex Ho, Rolf Neugebauer, Ian Pratt, and Andrew Warfield (2003)

Introduced paravirtualization, requiring hypervisor-aware modifications to guest OS kernels to hit near-native computation performance.

□

17. **The Multikernel: A New OS Architecture for Scalable Multicore Systems**

Andrew Baumann, Paul Barham, Pierre-Evariste Dagand, Tim Harris, Rebecca Isaacs, Simon Peter, Timothy Roscoe, Adrian Schüpbach, and Akhilesh Singhanian (2009)

The Barrelfish paper, treating cache-coherent multicore chips as a network of message-passing distributed nodes rather than a unified global shared memory pool.

□

18. **Non-Volatile Memory**

Steven Pelley, Thomas F. Wenisch, Brian T. Gold, and David Bridge (2014)

Analyzes the architectural impacts of placing fast persistent memory directly on the processor's memory bus, collapsing storage and volatile caches.

□

19. **OSV—An Operating System for the Cloud**

Avi Kivity, Dor Laor, Glauber Costa, Pekka Enberg, Nadav Har'El, Don Marti, and Vlad Zolotarov (2014)

Pioneered modern cloud "unikernels" by removing ancient multi-user isolation constructs to streamline runtime environments inside virtualized infrastructure.

□

20. Verifying a File System Implementation

Konstantine Arkoudas, Karen Zee, Viktor Kuncak, and Martin Rinard (2004)

Presents rigorous formal systems verification methods using simulation relations to mathematically validate structural correctness against data corruption.

□

Part VI: Selected Systems Research from Stony Brook University

Advanced research originating from labs at Stony Brook University (COMPAS, FSL, and associated collaborations), bridging hardware accelerators, modern microarchitecture, and file systems.

21. Clearing the Clouds: A Study of Emerging Scale-out Workloads

Michael Ferdman, Almutaz Adileh, Onur Kocatyali, Volkan Volkan, Jevgenijs Jaleckis, Djordje Jevdjic, Saurabh Kestur, Per Ekman, Adrianne Castro, Mark D. Hill, and Babak Falsafi (2012)

Winner of the ASPLOS Test-of-Time Award. Highlights the disconnect between conventional CPU architectures and modern data-center scales.

□

22. A Case for Hardware-Managed Virtual Memory on FPGAs

Zeljko Korolija, Timothy Roscoe, and Michael Ferdman (2020)

Investigates decoupling hardware accelerators from core kernel virtualization mechanics by placing memory management units directly inside FPGAs.

□

23. TailCheck: A Lightweight Heap Overflow Detection Mechanism

Seonmyeong So, Gyuwan Kim, and Michael Ferdman (2022)

Demonstrates hardware-software co-design using specialized structural tags to eliminate heap memory safety threats with negligible runtime overhead.

□

24. To FUSE or Not to FUSE: Performance of User-Space File Systems

Bharath Kumar Reddy Vangoor, Vasily Tarasov, and Erez Zadok (2017)

Provides a deep structural audit of performance degradation patterns and memory copying boundaries in standard user-space file systems layers.

□

25. Performance and Resource Utilization of FUSE User-Space File Systems

Vasily Tarasov, Bharath Kumar Reddy Vangoor, and Erez Zadok (2019)

Extends user-space layout profiling to show specific interaction mechanics during high context-switching and concurrent multi-threaded I/O workloads.

□

26. Evaluating Performance and Energy in File System Server Workloads

Priya Sehgal, Ming Chen, and Erez Zadok (2010)

Examines the tangible electrical footprints and efficiency boundaries of competing Linux storage layouts across intensive cloud servers.

□